Travel Reservation System

User Guide & Operating Instructions

Covers the web application in `TravelReservationSystem/` · Flask + SQLite · October 2026
For the technical internals, see `SYSTEM_DOCS.md` in the project root.

1. What this system is

It is a booking and ticketing system for a small transport operator. Customers sign up, choose a vehicle, enter where they are being picked up and where they are going, and the system works out the distance and a fare estimate. When the customer confirms, the booking is saved with that fare, seats are assigned automatically, and a printable ticket is issued per passenger. Admins control the vehicle types, their pricing rules, the vehicle fleet, and every booking.

Role	Where they work	What they can do
Customer	`/` login → `/dashboard` → `/reservation`	Register, book a trip, see the fare estimate, view and print tickets and receipts, edit or cancel their own bookings.
Admin	`/admin-login` → `/admin`	Set and change pricing per vehicle type, add new vehicle types, enable or disable types, register vehicles, see all reservations, tickets and users, delete bookings.

The key idea: **no price is written into the code or the web pages**. Every fare comes from the vehicle type's row in the database, so the operator can reprice anything — or add a completely new vehicle type — from the admin screens without touching a line of code.

2. Getting started

2.1 Requirements

- Python 3.13 (a `.venv` exists in the project root; the system Python has no Flask).
- Two packages at minimum: `Flask` and `qrcode` (plus `Pillow` for image output).

- An internet connection for address search, road-routing and map tiles. The system still works without it — see **§8 Troubleshooting**.

2.2 Start the server

```
cd Travel-reservation-project
python -m venv .venv
.venv/bin/pip install -r TravelReservationSystem/requirements.txt

cd TravelReservationSystem
../.venv/bin/python app.py
```

Then open **http://127.0.0.1:5000** in a browser. Other devices on the same network can use `http://<your-computer-ip>:5000`, since the server listens on all interfaces.

Item	Value
Customer entry point	`http://127.0.0.1:5000/`
Admin entry point	`http://127.0.0.1:5000/admin-login`
Default admin sign-in	username `admin` · password `admin123`
Database file	`TravelReservationSystem/travel_reservation.db` — created and upgraded automatically on start-up
First customer	Create one from **Register**; there is no seeded customer account.

Before going live: the `admin123` password and the session secret in `app.py` are development defaults. Change both, and run behind a real WSGI server (`gunicorn app:app`) with HTTPS instead of `app.py`'s built-in debug server.

3. Customer instructions

3.1 Create an account

1. Open the site and click **Register**.
2. Fill in **Full Name**, **Username**, **Contact Number**, **Password** and **Confirm Password**. The

username must not already be taken and the two passwords must match.

3. Sign in with that username and password. You land on the **Customer Dashboard**.

3.2 Book a trip

From the dashboard click **+ Make New Reservation**. The form has these fields:

Field	What to enter
Number of Passengers	How many people travel. This multiplies the fare for per-passenger vehicle types and limits how many you may book.
Passenger names	One name box appears per passenger. **Every box must be filled** — the count of names must equal the passenger count, or the booking is rejected.
Contact Number	Your phone, e.g. `09XXXXXXXXX`.
Vehicle Type	Dropdown. Each entry shows its own price rule, e.g. *Private Van — ₱1,500.00 base (first 25 km) · ₱45.00/km after 25 km · per booking*. Only vehicles that are both enabled and priced appear here.
Pickup / Starting Location	Type an address or place name and press 🔍 **Search**, or place a pin with 📍 **Set Pickup** on the map.
Destination	Same — 🔍 **Search** or 📍 **Set Dest**.
Travel Date / **Travel Time**	Date and time pickers.
Distance (km)	Normally left alone — it fills itself once the route is measured. It is the kilometres the fare is calculated from.

Recommended order: set the passenger count, pick the vehicle, set the pickup, set the destination, then click 🗺️ **Calculate Road Distance & Fare**. The map draws the road, the distance box fills in, and the fare panel updates. Changing the vehicle, the passenger count or either location recalculates immediately.

3.3 Read the fare panel

Below the form a panel lists the distance, the pricing rule of the chosen vehicle, each line of the

calculation, and the **Estimated Fare**. It is labelled as an estimate on purpose:

```
This is an estimate based on the route the map found.
The confirmed fare is the one saved with your booking.
```

Worked example — **Private Van** (₱1,500 base covering 25 km, ₱45 per extra km, charged once per booking) for a **40 km** trip:

```
extra distance = 40 − 25            = 15 km
distance charge = 15 × ₱45          = ₱675.00
trip fare = ₱1,500 + ₱675           = ₱2,175.00
charged once (per booking)          = ₱2,175.00
```

The same 40 km on the **Bus** (₱2.22/km, charged per passenger) with 3 passengers:

```
trip fare = 40 × ₱2.22              = ₱88.80
× 3 passengers                      = ₱266.40
```

3.4 Confirm, then collect the tickets

1. Check the vehicle, route, distance and total shown, then submit the form.
2. You are taken to a confirmation page showing the **booking reference** (`BKF-YYYYMMDD-NNNN`), the saved fare and the seats.
3. One ticket is created **per passenger**, numbered `TRS-000001-01`, `TRS-000001-02`, and so on.
4. From the dashboard, use 🎫 **View Ticket** for the printable ticket and 🧾 **Receipt** for the payment receipt. Each ticket carries a **QR code**; scanning it opens a public verification page showing that ticket's route, date, seat and status — useful for a driver or gate check.

3.5 Change or cancel a booking

On the dashboard, next to each reservation:

Button	Effect
✏️ **Edit**	Opens the booking pre-filled. Change passengers, vehicle, route, date or time. If you change the pickup or destination, the route is re-measured and the fare is recalculated; if the route stays the same, the saved distance is reused. The ticket fare shown after saving is the new computed fare.

🚫 **Cancel**	Marks the reservation and its tickets *Cancelled* and **frees the seats** so other customers can take them. Not available once a booking is *Completed* or already cancelled.
🗑️ **Delete**	Removes the booking permanently. Cancel instead if you want the record kept for the operator's accounting.

You can only touch bookings signed in under your own account.

4. Admin instructions

4.1 Sign in and find your way around

Go to `/admin-login` and sign in with the admin credentials. The sidebar gives you:

Section	Purpose
📊 Dashboard	Counters, plus the **Available Vehicle Types**, **Vehicle Registry**, **All Reservations**, **All Passenger Tickets** and **Registered Users** tables on one page.
⚙️ Vehicle Types	The pricing table — one row per type, with its fare rule and bookable status. **This is where fares are set.**
🚐 Vehicle Registry	The physical fleet: individual vehicles, plates, capacity, driver, status.
📋 All Reservations / 🎫 All Tickets	Every booking and every issued ticket, with links to the ticket and receipt.
👥 Registered Users	Customer accounts.

4.2 Set or change a vehicle type's pricing

On the dashboard, in **Available Vehicle Types**, click ✏️ **Edit Pricing** on a type (or open `/admin/vehicle-types` for the full list). The form is the same for every type:

Field	Meaning

Distance Pricing	
Base Fare (₱)	Flat charge for the trip, before any kilometres.
Included Distance (km)	Kilometres covered by the base fare. Beyond this, the additional rate applies. Set `0` for a pure per-kilometre fare.
Additional Rate (₱/km)	Price per kilometre *beyond* the included distance.
Minimum Fare (₱)	Floor price. Use `0` for no floor.
Additional Charges	
Amount (₱)	Optional flat add-on — terminal fee, toll, surcharge.
Label Shown to Customers	What that add-on is called on the quote and receipt, e.g. *Terminal fee*. Maximum 60 characters.
Who Is Charged	
Per booking	One price for the whole vehicle — charter style. Good for Private Van, Taxi.
Per passenger	The computed trip fare is multiplied by the passenger count — seat-sale style. Good for Bus, Van.
Capacity & Availability	
Maximum Passengers per Booking	Seat capacity, 1–500. Drives seat assignment and blocks over-booking.
Location Source	*Land — road addresses*, *Air* or *Sea*; describes how the route is looked up.
Available for booking	Offer this type to customers or hide it.

A live preview sits on the same page. As you type, it shows what the fare would be at 5 km, 50 km and 200 km with the values currently in the boxes — check those three figures before saving and you will catch almost any input mistake.

Validation rules the form enforces: amounts must be numeric and cannot be negative; passenger capacity must be between 1 and 500; and a type must have *at least one* charge set (a base fare, a per-kilometre rate, a minimum fare or an additional fee) before it will save. This is what stops a vehicle being bookable at ₱0.

> **Repricing is not retroactive.** Saving new rates affects future bookings only. Existing reservations keep the fare they were quoted, and their tickets and receipts keep showing that original amount.

4.3 The fare rule in one place

```
extra distance = max(0, total distance − included distance)
trip fare      = max(minimum fare, base fare + extra distance × additional rate)
total fare     = trip fare × passengers + additional charges   (per passenger)
               = trip fare               + additional charges  (per booking)
```

The minimum fare is applied **per trip, before** the passenger multiplier, and the additional charge is added **after** it. So a per-passenger type with a ₱400 floor and 3 passengers collects ₱1,200, not ₱400.

4.4 Add a brand-new vehicle type

1. On the dashboard click **+ Add Vehicle Type** (or go to `/admin/vehicle-types/new`).
2. Enter the **Vehicle Type Name** and the full pricing form in the same step.
3. Save. The type appears in the customer dropdown immediately — no code change, no restart.

A duplicate name is rejected with a message and your entries are kept so you can correct them. The name cannot be renamed afterwards; delete-and-recreate, or keep the type disabled, instead.

Nothing is bookable until it is priced. A type without fare settings is listed as **Needs pricing**, is excluded from the customer picker, and if you try to switch it on the system refuses and sends you to the fare form.

4.5 Enable or disable a type

Each row in the vehicle-type list has an on/off toggle. Turning a type **off** removes it from the customer picker at once but leaves existing bookings and tickets untouched. Turning it **on** is blocked while the type has no pricing.

4.6 Register vehicles in the fleet

🚐 **Vehicle Registry → + Register Vehicle.** Fields: **Vehicle Code** (auto-assigned when adding),

Vehicle Type (must be an existing type), **Vehicle Name / Model**, **Plate Number**, **Passenger Capacity** (1–200), **Assigned Driver** and **Vehicle Status**. This is the roster of actual units and their drivers — separate from the type, which holds the pricing.

4.7 Manage bookings as admin

All Reservations and **All Passenger Tickets** list every booking with its saved distance and fare, and link straight to the ticket and receipt for printing. Admins can delete a reservation from here. Use the ticket's QR page (`/verify/<ticket number>`) to check a passenger's ticket at boarding — it shows the live status, so a cancelled ticket reads as cancelled.

5. Current fare card

As configured in this database today. Change these from the admin screens in §4.2; this table is only a snapshot.

Type	Base	Incl. km	₱/km	Charged	Seats	Status
Bus	₱0	0	₱2.22	per passenger	45	Bookable
Van	₱0	0	₱2.00	per passenger	12	**Disabled** — hidden from customers
Private Van	₱1,500	25	₱45.00	per booking	10	Bookable
Taxi	₱0	0	₱13.50	per passenger	4	Bookable

No type currently sets a minimum fare or an additional charge, so both are 0 across the board. Add either from the fare form and it appears here.

Sample fares at 40 km

Type	1 passenger	3 passengers	10 passengers
Bus	₱88.80	₱266.40	₱888.00
Private Van	₱2,175.00	₱2,175.00	₱2,175.00

Taxi	₱540.00	₱1,620.00	over capacity (4 seats)

6. How distance is measured

1. Pickup and destination are turned into coordinates by OpenStreetMap's **Nominatim** search (results are cached, so repeat lookups are fast).
2. With both coordinates known, the system asks **OSRM** for the *driving route* and uses that road distance — the route line is drawn on the map.
3. If routing is unavailable, it falls back to the straight-line distance between the two points.
4. The distance is sanity-checked server-side against the straight-line figure and capped at 2,000 km, so a mistyped or tampered value cannot produce an absurd fare.

Road distance is normally 1.2–1.6× the straight-line distance, so an estimate from a straight-line fallback reads slightly low. Re-run 🗺️ **Calculate Road Distance & Fare** when the map is reachable.

7. Seats and ticket numbering

Seats are handed out automatically. A "trip" is the combination of pickup location, origin, destination, date, time and vehicle; the first free seats in that trip are taken, and seats from cancelled tickets become available again. If the trip is full, the booking is refused.

Type	Seat labels
Bus	`A1 ... A45`
Van	`V1 ... V12`
Private Van	`PV1 ...` up to the capacity you set
Taxi	`T1 ... T4`
Any type you add	`S1 ...` up to the capacity you set — generic labels, no layout needed

Booking references look like `BKF-20261018-0007`; ticket numbers like `TRS-000042-03` (reservation 42, third passenger).

8. Troubleshooting

Symptom	Cause and fix
Search returns nothing / map pins never appear	Nominatim is unreachable or rate-limited (its public policy allows about one request per second). Wait a moment and retry, or place the pin manually with 📍 **Set Pickup** / 📍 **Set Dest**.
Distance stays blank after searching	The OSRM routing server did not answer. Click 🗺️ **Calculate Road Distance & Fare** again; if it keeps failing the system falls back to straight-line distance once both points are set.
"The number of passenger names must match the number of passengers."	Fill every passenger name box, or lower the passenger count.
A vehicle type is missing from the dropdown	It is either switched off or has no pricing yet. As admin, use **Edit Pricing** to set rates and the toggle to switch it on.
"... cannot be booked yet: an admin has to set its fare first."	Same as above — the type exists but is unpriced, so the system refuses to guess a number.
The fare form will not save	A negative or non-numeric amount, a capacity outside 1–500, or every charge left at zero. Set at least one charge.
Quoted fare differs from the saved fare	Should no longer happen — distance is carried at two decimals end to end. If you edited the distance box by hand, click 🗺️ **Calculate Road Distance & Fare** to re-measure.
"does not have enough available seats"	That trip is full. Change the time, or cancel a booking to release seats.
Edited a price but an old ticket still shows the old amount	Correct behaviour. Bookings keep the fare they were quoted.
`ModuleNotFoundError: flask`	You are using the system Python. Activate the project's `.venv` first.

Browser says the site is unreachable on an odd port	Chrome blocks some ports as unsafe (5061 and 5062 among them). Use 5000, or 8090 for testing.

9. Things to know before real use

- **Fares are estimates up to the moment of booking.** The booking screen carries that notice; once confirmed, the amount saved with the reservation is what the ticket and receipt show, and it does not drift when prices are later changed. Traffic, tolls and route changes are not tracked automatically; use the edit screen (§3.5) to reprice a booking through the normal flow rather than editing the database.
- **Free-tier map services.** Nominatim and the OSRM demo server are public, rate-limited and carry no availability guarantee. For a real operation, self-host or buy a routing/geocoding key.
- **Passwords are stored as plain SHA-256 hashes**, which is not a modern hardened scheme, and the session secret is a literal in `app.py`. Fix both before this faces customers.
- **Admin screens rely on the session flag** set at admin login; keep the admin password private, since pricing authority comes from that account.
- **SQLite** suits a single-site operator with modest volume. Many simultaneous writers would call for a client/server database.
- **Legacy files** — `app_backup.py`, `TRAVEL SYSTEM.py`, `templates/reservation_backup.html`, and the `main.py` desktop prototype at the project root — are not part of the running web app. Do not edit them expecting a change on screen.

10. Quick reference

I want to…	Go to
Book a trip	Dashboard → **+ Make New Reservation** → `/reservation`
See my tickets	Dashboard → **My Tickets** → 🎫 **View Ticket**
Print a receipt	Dashboard → **My Tickets** → 🧾 **Receipt**
Cancel a booking	Dashboard → 🚫 **Cancel**

Change a fare rule	`/admin` → Available Vehicle Types → ✏️ **Edit Pricing**
Add a new vehicle type	`/admin` → **+ Add Vehicle Type**
Switch a type on/off	`/admin/vehicle-types` → row toggle
Register a vehicle / driver	`/admin/vehicles` → **+ Register Vehicle**
Verify a passenger's ticket	Scan the QR, or open `/verify/TRS-000042-01`
See every booking or ticket	`/admin` → All Reservations / All Passenger Tickets

Travel Reservation System — User Guide · web app in `TravelReservationSystem/` · technical reference: `SYSTEM_DOCS.md`